PROVA 1 — REVISÃO

Projeto e Análise de Algoritmos

Professor: Críston Pereira de Souza **Data:** 23/09/2026

Aluno(a): _____

---

1. Contagem de execuções **[1,25 ponto(s)]**

O algoritmo abaixo recebe duas listas: A, com n elementos, e B, com m elementos. Considere o pior caso em que a condição da linha 6 é sempre verdadeira. Para cada linha, forneça o número de execuções em função de n e m. Mantenha somatórios quando conveniente. Em seguida, forneça a simplificação Θ dos somatórios.

```
1  def combina(A, B):
2      total = 0
3      for i in range(n):
4          total += A[i]
5          for j in range(m - i):
6              if B[j] >= 0:
7                  total += B[j]
8      return total
```

Assuma $m \geq n$.

```
Linha 1: 1 execução
Linha 2: 1 execução
Linha 3: n+1 execuções
Linha 4: n execuções
Linha 5: \sum_{i=0}^{n-1} (m - i + 1) = nm - \sum_{i=0}^{n-1} (i+1)
         = nm - Theta(n^2) = Theta(nm)
Linha 6: \sum_{i=0}^{n-1} (m - i) = nm - \sum_{i=0}^{n-1} i
         = nm - Theta(n^2) = Theta(nm)
Linha 7: igual linha 6, portanto Theta(nm) execuções
Linha 8: 1 execução
```

2. Notação assintótica e simplificação **[1,25 ponto(s)]**

a) Considere

$$f(n) = 7 \cdot 8^n n^2 \log n, \qquad g(n) = 3 \cdot 4^n n^{20}.$$

Indique qual função cresce mais.

b) Simplifique em notação Θ:

$$S(n) = \sum_{i=1}^{n} i^3 \log^2 i.$$

c) Um inteiro positivo N ocupa no máximo b bits. Determine, em notação Θ, o número máximo de iterações de:

```
x = N
while x > 1:
    x = x // 2
```

em função de b.

```
a) f(n) tem parte exponencial com maior base, portanto f(n) cresce mais
   rapidamente que g(n).
b) S(n) = sum_{i=1}^{n} i^3 log^2 i = Theta(n^4 log^2 n)
   Aritmética: Theta(n f(n)).
c) A cada iteração, x é aproximadamente dividido por 2.
   Logo, o número de iterações é Theta(log N). Como N < 2^b, temos
   Theta(log 2^b) = Theta(b).
```

3. Divisão e conquista **[1,25 ponto(s)]**

Considere a recorrência

$$T(n) = 9T(n/3) + \Theta(n^2 \log n), \qquad T(1) = \Theta(1).$$

a) Identifique a, b, d e e na forma

$$T(n) = aT(n/b) + \Theta(n^d \log^e n).$$

b) Compare d com $\log_b a$.

c) Resolva a recorrência em notação Θ.

```
a) a=9, b=3, d=2, e=1
b) log_b a = log_3 9 = 2, portanto d = log_b a
c) No caso crítico com e=1, acrescenta-se uma potência de log:
T(n) = Theta(n^2 log^2 n)
```

4. Decremento e conquista **[1,25 ponto(s)]**

Para cada recorrência, identifique a, b e $f(n)$ no modelo $T(n) = aT(n-b)+f(n)$ e determine a solução em notação Θ.

a) $T(n) = T(n-3) + \Theta(n^2 \log n)$.

b) $R(n) = 2R(n-5) + \Theta(n^4)$.

```
a) a=1, b=3, f(n)=Theta(n^2 log n)
   Theta(n f(n)) = Theta(n^3 log n)
b) a=2, b=5, f(n)=Theta(n^4)
   Theta(a^(n/b)) = Theta(2^(n/5))
```

5. Análise empírica por regressão linear **[1,25 ponto(s)]**

Adote o modelo

$$T(n) \approx c\, 10^{an} n^d (\log_{10} n)^e$$

e a regressão linearizada

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3.$$

Uma regressão produziu

$$\beta_0 = -5, \qquad \beta_1 = 0{,}20, \qquad \beta_2 = 2, \qquad \beta_3 = -1.$$

a) Determine c, a, d e e.
b) Escreva a função empírica $T(n)$ e sua simplificação em notação Θ.
c) Para uma medição com $n = 100$ e $T(100) = 10^{-1}$ s, escreva Y, X_1, X_2, X_3.

---

a) c = 10^{-5}, a = 0.20, d = 2, e = -1
b) T(n) ~ 10^{-5} 10^{0.20n} n^2 / log n
   T(n) in Theta(10^{0.20n} n^2 / log n)
c) Y = -1, X_1 = 100, X_2 = log10(100) = 2, X_3 = log10(2) ~ 0.3010

6. Razões assintóticas **[1,25 ponto(s)]**

a) Para um algoritmo com fator exponencial, foram medidos:

n	20	21	22
$T(n)$	0,40	1,13	3,20

Calcule as duas razões consecutivas. Admitindo $T(n) \approx c\, 2^{an} n^d \log_2^e n$, estime 2^a.

b) Após remover um eventual fator exponencial de outro algoritmo, obteve-se $R(n) \approx c\, n^d$:

n	500	1000	2000
$R(n)$	0,50	2,00	8,00

Use a razão dobrando para estimar d e, com $n = 2000$, estime c. Utilizando estes parâmetros, estime o tempo de execução para n = 4000.

---

a) Razões consecutivas:
T(21)/T(20) = 1.13/0.40 = 2.825
T(22)/T(21) = 3.20/1.13 ~ 2.832
Estimativa: 2^a ~ 2.8
b) Razão dobrando:
R(1000)/R(500) = 2.00/0.50 = 4
R(2000)/R(1000) = 8.00/2.00 = 4
Logo, 2^d = 4 => d = 2
Usando n=2000:
c ~ 8 / 2000^2 = 2 * 10^{-6}
Estimando para n=4000:
R(4000) ~ 2 * 10^{-6} * 4000^2 = 32

7. TADs e escolha de implementação **[1,25 ponto(s)]**

Analise a função a seguir, que gerencia o fluxo de uma fila de impressão:

```
def processa_impressao(documentos):
    colecao = cria_colecao_vazia()
    for doc in documentos:
        colecao.insere_inicio(doc)

    ordem_saida = []
    while not colecao.esta_vazia():
        atual = colecao.remove_final()
        ordem_saida.append(atual)
    return ordem_saida
```

Tarefa:

1. Conte o número de execuções das operações da coleção para uma entrada com n documentos.
2. Identifique a melhor estrutura vista em aula para essa finalidade e explique por que o uso ingênuo de array dinâmico comum (`appendleft`) tornaria a solução ineficiente.
3. Calcule a complexidade total final com a estrutura recomendada e com o vetor comum.

a) A função insere cada documento no início da coleção, portanto são n execuções de insere_inicio. Em seguida, a função remove cada documento do final da coleção, portanto são n execuções de remove_final.
b) A melhor estrutura para essa finalidade é uma deque (lista duplamente encadeada). O uso ingênuo de um array dinâmico comum com appendleft seria ineficiente porque a operação de inserir no início de um array dinâmico requer deslocar todos os elementos subsequentes, resultando em O(n) para cada inserção, tornando a complexidade total O(n^2).
c) Com a deque, cada operação de inserção e remoção é O(1), resultando em uma complexidade total de O(n). Com o array comum, a complexidade total seria O(n^2) devido ao custo de deslocamento dos elementos em cada appendleft.

---

8. Cartões Anki: complexidades **[1,25 ponto(s)]**

Para cada operação, indique a complexidade assintótica correspondente.

a) Inserir em posição arbitrária em array dinâmico

b) Encontrar a menor chave de um dicionário com tabela hash

c) União $A \cup B$ de conjuntos com tabela hash

d) Consultar o menor elemento de uma heap binária, sem removê-lo

e) Iterar pelos vizinhos de um vértice u em matriz de adjacências

a) O(n)
b) O(n)
c) O(|A|+|B|)
d) O(1)
e) O(n)